

Soccer TIPS: Engineering Department Document: Validation & Data Ingestion Handoff Summary

Part One: SkillCorner Open Data Validation, Constraints, and Reusable Ingestion Guidance

1. Document Purpose

This document serves as the primary engineering handoff for the validated SkillCorner Open Data workflow used in the Soccer TIPS project.

The goal of this document is to consolidate:

- validated dataset behavior,
- ingestion rules,
- discovered constraints,
- structural findings,
- engineering recommendations,
- implementation warnings,
- reusable ingestion guidance,
- and future considerations

into a single source-of-truth handoff for downstream engineering work.

This document is intentionally broader than a normal “validation report.” It combines:

validation findings

+

engineering implications

+

recommended ingestion architecture direction

because the project is currently at a scale where separating those concerns would create unnecessary fragmentation.

This document should be treated as:
the authoritative Layer 1 ingestion and validation reference
for the SkillCorner portion of the project.

2. Intended Audience

This document is primarily intended for:

Lead Developer

To guide reusable ingestion module development.

Project Manager

To confirm Layer 1 readiness and engineering transition status.

Journalist / Documentation Assistant

To accurately document technical milestones and decisions.

Future Development Work

To preserve discovered assumptions and reduce future debugging confusion.

Portfolio / Interview Context

To demonstrate disciplined engineering and validation workflow rather than simplistic “load-and-plot” analytics work.

3. Project Context

Project Name

Soccer TIPS

Tactical Intelligence & Performance System

Project Goal

Build a tracking-data-driven football analytics system capable of:

- measuring tactical identity,
- computing spatial-temporal metrics,
- segmenting tactical phases,
- generating structured tactical insights,
- and eventually supporting AI-assisted interpretation and recommendations.

Current Scope

This handoff only covers:

Layer 1

Data Acquisition, Ingestion, Validation, and Preparation

This work does NOT yet include:

- feature engineering,
 - tactical metrics,
 - compactness modeling,
 - transition analysis,
 - machine learning,
 - AI interpretation,
 - recommendation systems,
 - dashboards,
 - or reporting layers.
-

4. Dataset Acquired

Primary Dataset

SkillCorner Open Data

Dataset Type

Broadcast-derived football tracking data with contextual tactical layers.

Competition

Australian A-League
2024/2025

Match Count

10 matches

Project Location

C:\Users\amsch\. Personal Work\Projects\Soccer TIPS

SkillCorner Repository Location

external/skillcorner/

5. Core Architectural Conclusion

The SkillCorner Open Data has been successfully validated enough to begin:

reusable ingestion engineering

The dataset is structurally usable.

The project has successfully moved from:

raw exploratory inspection

to:

validated ingestion engineering readiness

The next stage should focus on:

- reusable loaders,
 - standardized schemas,
 - processed tracking tables,
 - validation automation,
 - and ingestion module architecture.
-

6. Critical Engineering Findings

This section contains the most important validated rules discovered during exploration.

6.1 Match Discovery Rule

The file:

`data/matches.json`

contains the master list of available matches.

Each match row contains:

`id`

which maps directly to:

`data/matches/{match_id}/`

Example:

`match_id = 2017461`

`folder = data/matches/2017461/`

This relationship was validated successfully.

6.2 Expected Match File Structure

Expected files per match:

```
{id}_match.json  
{id}_tracking_extrapolated.jsonl  
{id}_phases_of_play.csv  
{id}_dynamic_events.csv
```

6.3 Known Dataset Issue

One match is missing dynamic events:

```
match_id = 1953632  
missing file = 1953632_dynamic_events.csv
```

This is the only confirmed structural dataset issue discovered so far.

Engineering implication

Dynamic events MUST be treated as optional.

Future ingestion code must NOT fail if:

```
dynamic_events.csv
```

is missing.

Recommended handling:

```
if dynamic_events_exists:  
    load_events()  
else:  
    events_df = None  
    log_warning()
```

6.4 Match Metadata Structure

Validated metadata fields include:

id
home_team_score
away_team_score
date_time
stadium
home_team
away_team
players
match_periods
home_team_side
pitch_length
pitch_width
ball

Important findings:

Pitch dimensions are dynamic

Observed:

pitch_length = 105
pitch_width = 68

DO NOT hardcode pitch dimensions globally.

Always preserve per-match pitch dimensions.

Home team attack direction changes by period

Observed:

home_team_side:
['left_to_right', 'right_to_left']

This will matter later for orientation normalization.

6.5 Player Metadata Structure

The player metadata table includes:

id
team_id
short_name
player_role
number
start_time
end_time
trackable_object
team_player_id

Important nested structure:

player_role

contains:

position_group
name

Example:

position_group = Central Defender
name = Right Center Back

This can later support:

- role-based metrics,
- tactical shape reconstruction,
- line grouping,
- positional clustering.

6.6 MOST IMPORTANT VALIDATION FINDING

Correct Player Join

This was the single most important discovery from the validation phase.

CORRECT JOIN

tracking player_data.player_id
→ players_df["id"]

INCORRECT JOINS

players_df["trackable_object"]
players_df["team_player_id"]

Validation results:

Matches players_df['id']:
True 22

Matches players_df['team_player_id']:
False 22

Matches players_df['trackable_object']:
False 22

Engineering implication

ALL tracking/player joins should use:

left_on="player_id"
right_on="id"

This rule should be considered foundational and non-negotiable unless later evidence disproves it.

6.7 Tracking File Structure

Tracking files are:

JSONL

NOT normal JSON.

Each line represents ONE tracking frame.

Correct parsing approach

```
with open(file, "r", encoding="utf-8") as f:  
    for line in f:  
        frame = json.loads(line)
```

DO NOT attempt to load the entire file as one JSON object.

6.8 Tracking Frame Structure

Validated frame keys:

```
frame  
timestamp  
period  
ball_data  
possession  
image_corners_projection  
player_data
```

6.9 Player Tracking Structure

Each player entry contains:

```
x  
y  
player_id  
is_detected
```

Example:

```
{  
  "x": -20.78,  
  "y": 3.31,  
  "player_id": 51013,  
  "is_detected": True  
}
```

6.10 Ball Tracking Structure

Ball fields:

x
y
z
is_detected

Ball positions appear valid and align with pitch coordinates.

6.11 Empty Frame Behavior

Tracking frames are NOT partially sparse.

This was a major discovery.

The dataset behaves almost entirely as:

either:

0 players

or:

22 players

Observed distribution for match 2017461:

0 players 31047 frames

22 players 40404 frames

There were essentially no partially populated frames.

Engineering implication

This is much cleaner than expected.

Initial usable-frame logic should be:

```
len(frame["player_data"]) == 22
```

6.12 Tracking Volume Findings

Example match:

match_id = 2017461

Validation summary:

total_frames = 71451

usable_22_player_frames = 40404

At 10 fps:

$40404 / 10 \approx 67$ minutes

This is realistic for active in-play usable tracking coverage.

6.13 Broadcast Tracking Limitation

This is BROADCAST tracking, not stadium optical tracking.

Implications:

- camera cuts exist,
- off-screen players exist,
- extrapolation exists,
- detection confidence varies.

This explains:

- empty frames,
- extrapolated players,
- and detection flags.

This is NOT evidence of broken data.

6.14 Detection Flag Findings

Validated field:

is_detected

Observed example:

True = directly detected

False = extrapolated

Example frame:

18 detected

4 extrapolated

Engineering implication

DO NOT remove `is_detected`.

Preserve it in processed outputs.

Potential future uses:

- confidence scoring,
- filtering,
- quality diagnostics,
- uncertainty weighting.

6.15 Coordinate System Findings

Coordinates are:

metric

centered at midfield

For a 105 x 68 pitch:

$x \approx -52.5$ to $+52.5$

$y \approx -34$ to $+34$

Visual inspection confirmed realistic football structure.

6.16 Visual Validation Result

A reconstructed frame successfully produced:

- realistic kickoff-like positioning,
- correct team separation,
- realistic goalkeeper depth,
- ball near midfield,
- no obvious coordinate corruption.

This strongly validated:

- joins,
 - coordinate realism,
 - frame parsing,
 - and overall structural usability.
-

6.17 Possession Field Status

Tracking frames include:

possession

but this field has NOT yet been deeply validated.

Observed first usable frame:

```
{  
  "player_id": None,  
  "group": None  
}
```

Engineering implication

Possession fields should NOT yet be treated as authoritative.

Future validation should compare:

- tracking possession,
- phases,
- dynamic events.

For now:

- phases are the more trustworthy segmentation layer.
-

7. Phases of Play Findings

The phases file was one of the strongest discoveries for the future project.

7.1 Phase Structure

Validated columns include:

frame_start
frame_end
period
team_in_possession_id
team_in_possession_phase_type
team_out_of_possession_phase_type

7.2 Phase Alignment

Validation confirmed:

phase_start_before_tracking = 0
phase_end_after_tracking = 0
invalid_phase_ranges = 0

Meaning:

- phase windows align cleanly to tracking frames.
-

7.3 Tactical Context Findings

Validated in-possession phases:

build_up
create
finish
direct
transition
quick_break
set_play
chaotic

Validated out-of-possession phases:

high_block
medium_block
low_block
defending_transition
defending_direct
defending_set_play
defending_quick_break
chaotic

7.4 Strategic Importance

This is a major acceleration point for the project.

The dataset ALREADY provides:

- tactical segmentation,
- possession context,
- defensive structure context.

This aligns extremely well with the project's intended tactical pipeline.

It may significantly reduce:

- manual segmentation work,
 - heuristic phase reconstruction,
 - and custom transition labeling.
-

8. Dynamic Events Findings

8.1 Dynamic Events Scale

Example match:

4188 rows

294 columns

This is a very rich contextual event dataset.

8.2 Dynamic Events Alignment

Validation confirmed:

events_before_tracking = 0

events_after_tracking = 0

invalid_event_ranges = 0

Meaning:

- event windows align cleanly to tracking frames.
-

8.3 Sparse Column Interpretation

Many columns contain high missing counts.

This is expected.

The dataset is:

- event-specific,
- context-specific,
- and wide.

Missing values DO NOT indicate corruption.

Most sparse columns apply only to:

- certain event types,
- tactical situations,
- possession contexts.

8.4 High-Value Event Context Fields

Observed contextual concepts include:

line-breaking
pressing chains
xThreat-style fields
pass reception matching
run subtypes
possession matching
fully extrapolated indicators

This dataset is much richer than standard event logs.

9. Multi-Match Consistency Findings

Multi-match sanity checks on the first three matches showed consistent structure.

Observed:

~70k tracking frames per match
~40k–48k usable 22-player frames
~400–500 phase rows

This strongly suggests:

- stable dataset formatting,
 - stable frame behavior,
 - stable phase structures across matches.
-

10. Recommended Reusable Ingestion Direction

The project should now move from:

exploration

to:

formal ingestion engineering

10.1 Recommended Folder Structure

src/data/

Suggested files:

load_skillcorner.py
validate_skillcorner.py
schemas.py

10.2 Recommended Loader Functions

load_matches_index()
load_match_metadata(match_id)

```
load_players(match_id)
load_tracking_frames(match_id)
load_phases(match_id)
load_dynamic_events(match_id)
validate_match_files(match_id)
validate_tracking_structure(match_id)
validate_phase_alignment(match_id)
validate_event_alignment(match_id)
```

10.3 Recommended Processed Tracking Format

Recommended long-format tracking schema:

```
match_id
frame
timestamp
period
team_id
player_id
short_name
position_group
role_name
x
y
is_detected
```

Recommended separate ball table:

```
match_id
frame
timestamp
period
ball_x
ball_y
ball_z
ball_is_detected
```

10.4 Recommended Validation Output

Each ingestion run should generate:

```
{  
  "match_id": match_id,  
  "tracking_total_frames": ...,  
  "usable_22_player_frames": ...,  
  "phase_rows": ...,  
  "event_rows": ...,  
  "player_join_success": ...,  
  "known_issues": ...  
}
```

11. Important Things NOT Yet Validated

The following remain future tasks:

- orientation normalization
- possession reliability
- tracking smoothing
- speed/acceleration smoothing
- frame interpolation
- cross-match orientation consistency
- phase reliability benchmarking
- detection-confidence filtering

These belong to later engineering and analytics layers.

12. What Should Happen Next

The next correct step is:

Lead Developer converts notebook validation logic into reusable ingestion modules.

NOT:

- tactical metrics,
- machine learning,
- AI interpretation,
- dashboards,
- recommendation systems.

The project is NOT ready for those yet.

The project IS ready for:

clean reusable ingestion engineering

13. Final Engineering Conclusion

The SkillCorner Open Data has successfully passed initial structural validation for the Soccer TIPS project.

The dataset is:

- structurally consistent,
- tactically rich,
- frame-aligned,
- and engineering-ready.

The most important validated rules are:

tracking player_id joins to players_df["id"]
usable tracking frames contain 22 players
dynamic events are optional
phases align to tracking frames
event windows align to tracking frames
coordinates are metric and centered at midfield
is_detected must be preserved

The most important engineering implication is:

The project should now transition from exploratory notebooks to reusable ingestion infrastructure.

Part Two: Metrica Open Data Validation, Constraints, and Reusable Ingestion Guidance

1. Document Purpose

This document serves as the primary engineering handoff for the validated Metrica Sports sample data workflow used in the Soccer TIPS project.

The goal of this document is to consolidate:

- validated dataset behavior,
- ingestion rules,
- structural findings,
- provider-specific constraints,
- engineering implications,
- reusable ingestion guidance,
- and future architectural considerations

into a single source-of-truth handoff for downstream engineering work.

This document intentionally combines:

validation findings

+

engineering implications

+

provider-specific ingestion guidance

because the project is currently transitioning from:

exploratory validation

to:

reusable ingestion infrastructure development

This document should be treated as:
the authoritative Layer 1 ingestion and validation reference
for the Metrica Sports sample dataset portion of the project.

2. Intended Audience

This document is primarily intended for:

Lead Developer

To guide reusable ingestion architecture and provider-specific loader development.

Project Manager

To confirm Layer 1 readiness and multi-provider ingestion status.

Journalist / Documentation Assistant

To accurately document technical validation milestones and engineering discoveries.

Future Development Work

To preserve provider-specific assumptions and reduce future ingestion/debugging confusion.

Portfolio / Interview Context

To demonstrate multi-provider data engineering workflow, validation methodology, and ingestion architecture planning.

3. Project Context

Project Name

Soccer TIPS
Tactical Intelligence & Performance System

Project Goal

Build a tracking-data-driven football analytics system capable of:

- measuring tactical identity,
- computing spatial-temporal metrics,
- analyzing transitions and compactness,
- generating structured tactical insights,
- and eventually supporting AI-assisted tactical interpretation.

Current Scope

This document only covers:

Layer 1

Data Acquisition, Ingestion, Validation, and Preparation

This work does NOT yet include:

- feature engineering,
 - tactical metrics,
 - compactness modeling,
 - transition stability modeling,
 - machine learning,
 - AI interpretation,
 - dashboards,
 - or reporting systems.
-

4. Dataset Acquired

Dataset Name

Metrica Sports Sample Data

Dataset Type

Research/demo football tracking and event data.

Repository Location

external/metrica/

Data Folder

external/metrica/data/

Validated Sample Folders

Sample_Game_1

Sample_Game_2

Sample_Game_3

5. Core Architectural Conclusion

The Metrica Sports sample data has been successfully validated enough to begin:

provider-specific reusable ingestion engineering

The dataset is structurally usable and significantly cleaner than the SkillCorner dataset from an ingestion perspective.

The dataset is NOT intended to replace SkillCorner as the project's primary tactical dataset.

Instead, Metrica serves as:

a clean engineering and ingestion benchmark dataset

while SkillCorner serves as:

the richer real-world tactical dataset

The combination of both datasets significantly strengthens:

- ingestion architecture robustness,
- provider generalization,

- and future engineering credibility.
-

6. Internal Dataset Structure Findings

The Metrica repository contains two different internal dataset formats.

6.1 Sample Game 1 & 2 Structure

CSV-based structure:

RawEventsData.csv
RawTrackingData_Home_Team.csv
RawTrackingData_Away_Team.csv

6.2 Sample Game 3 Structure

Alternative structure:

events.json
metadata.xml
tracking.txt

6.3 Engineering Implication

Metrica does NOT use one universal internal schema.

Reusable ingestion architecture should therefore treat:

Sample_Game_1 / Sample_Game_2

and:

Sample_Game_3

as separate ingestion pipelines if Sample Game 3 is later integrated.

Current validation focused primarily on:

Sample_Game_1

Sample_Game_2

because they represent the classic Metrica football tracking structure most commonly used in analytics workflows.

7. Tracking File Structure Findings

Unlike SkillCorner tracking:

one row $\neq$ one player

Metrica tracking uses:

one row = one frame

with all player coordinates embedded across many columns.

This is one of the largest provider-level architectural differences discovered during validation.

7.1 Metadata Header Structure

Tracking CSV files contain metadata rows before true tracking data begins.

Validated structure:

Row 0 = team labels

Row 1 = player/jersey identifiers

Row 2 = true column names

Row 3+ = actual tracking frames

Correct parser:

```
pd.read_csv(file, skiprows=2)
```

Using:

```
skiprows=3
```

incorrectly shifts the first tracking frame into column headers.

This was validated directly during parsing.

7.2 Tracking Schema Findings

Validated core columns:

Period

Frame

Time [s]

Player coordinates exist in paired columns.

Example:

Player11

Unnamed: 4

Player1

Unnamed: 6

...

Meaning:

named column = x coordinate

following unnamed column = y coordinate

Ball coordinates appear in the final paired columns:

Ball

Unnamed: 32

7.3 Wide-to-Long Engineering Requirement

Metrica tracking is inherently:

wide format

while the Soccer TIPS ingestion architecture is expected to standardize around:

long-format tracking tables

Therefore Metrica ingestion will require:

- column pairing logic,
- player extraction logic,
- and wide-to-long transformation logic.

This is a major provider-specific ingestion requirement.

8. Coordinate System Findings

Metrica coordinates are normalized rather than metric.

Observed coordinate ranges:

-0.05 → 1.06

Meaning:

- pitch coordinates are approximately normalized to $[0, 1]$,
 - with small overflow tolerance outside pitch boundaries.
-

8.1 Provider Coordinate Comparison

SkillCorner:

metric coordinates
centered at midfield

Metrica:

normalized pitch coordinates

8.2 Engineering Implication

Future multi-provider workflows will likely require:

provider normalization layer

to standardize coordinate systems before:

- compactness calculations,
 - centroid calculations,
 - tactical clustering,
 - and spatial-temporal modeling.
-

9. Tracking Continuity Findings

Tracking continuity was extremely clean.

Validated findings:

no duplicate frames
no frame gaps
clean frame continuity
perfect home/away alignment

Validation examples:

home_frame_duplicates = 0
away_frame_duplicates = 0
home_frame_gaps = 0
away_frame_gaps = 0

This dataset is structurally cleaner than SkillCorner.

9.1 Sampling Rate Findings

Validated sampling rate:

25 FPS

derived from:

Time [s] step size = 0.04

Provider comparison:

SkillCorner:

10 FPS

Metrica:

25 FPS

9.2 Engineering Implication

Future provider harmonization may require:

- temporal normalization,
 - resampling,
 - or provider-aware metric pipelines.
-

10. Event Dataset Findings

Metrica events are significantly simpler than SkillCorner dynamic events.

Validated event structure:

1745 rows
14 columns

Validated columns:

Team
Type
Subtype
Period
Start Frame
Start Time [s]
End Frame
End Time [s]
From
To
Start X
Start Y
End X
End Y

This resembles a traditional football event provider structure.

10.1 Event Alignment Findings

Validation confirmed:

events_before_tracking = 0
events_after_tracking = 0

Meaning:

- events align properly with tracking frames.

This passed validation successfully.

10.2 Invalid Event Range Finding

One invalid event range was discovered:

invalid_event_ranges = 1

The row was:

SET PIECE

KICK OFF

Start Frame = 1

End Frame = 0

This appears to be:

- kickoff initialization behavior,
- annotation bookkeeping behavior,
- or a non-critical event edge case.

This is NOT considered meaningful corruption.

10.3 Event Type Findings

Validated event types include:

PASS

RECOVERY

BALL LOST

CHALLENGE

SET PIECE

BALL OUT

SHOT

FAULT RECEIVED

CARD

Compared to SkillCorner:

- Metrica events are significantly simpler,
 - less contextual,
 - and less tactically rich.
-

11. Missingness Findings

Tracking tables contain substantial missing values.

Observed example:

~983k missing values

This is expected due to:

- substitutions,
- inactive players,
- partial tracking availability,
- and wide-format schema structure.

Certain player columns were heavily sparse:

Player14

Player13

Player1

Player12

This does NOT currently appear to indicate corruption.

12. Structural Comparison to SkillCorner

Validation reinforced that the two providers serve different project roles.

12.1 SkillCorner

Strengths:

- tactical richness,
- phases of play,
- contextual dynamic events,
- production-style tracking realism,

- richer football intelligence layers.

Weaknesses:

- noisier tracking,
 - extrapolation complexity,
 - lower FPS,
 - more complex ingestion.
-

12.2 Metrica

Strengths:

- extremely clean frame continuity,
- simpler engineering structure,
- ideal ingestion benchmark dataset,
- strong research/demo environment.

Weaknesses:

- less tactical context,
 - normalized coordinates,
 - wide-format complexity,
 - limited dataset scope.
-

13. Strategic Engineering Conclusion

The combination of:

SkillCorner

+

Metrica

is significantly stronger than either dataset alone.

The project now contains:

one production-style provider

+

one research/demo provider

This significantly improves:

- ingestion architecture robustness,
- provider-generalization capability,
- engineering realism,
- and future portfolio/interview credibility.

The project is now beginning to resemble:

multi-provider football analytics infrastructure

rather than:

single-dataset exploratory analysis

14. Engineering Readiness Conclusion

Metrica validation is considered complete enough for:

Layer 1 ingestion engineering readiness

The most important validated rules are:

tracking uses wide format

correct parser uses skiprows=2

coordinates are normalized

tracking continuity is extremely clean

events align properly with tracking

player coordinates exist in x/y column pairs

ball coordinates are embedded directly in tracking tables

25 FPS sampling rate is used

The next engineering step should be:

integration of Metrica ingestion logic into the reusable ingestion architecture

Part Three: Developer Implementation Contract

This section defines the required implementation rules for all future Layer 1 ingestion engineering work within the Soccer TIPS project.

The purpose of this section is to make provider handling, schema expectations, validation behavior, and architectural boundaries explicit before reusable ingestion infrastructure development begins.

This section should be treated as:

mandatory engineering guidance for all future ingestion-related development work.

1. Canonical Multi-Provider Tracking Schema

All providers should ultimately ingest into a shared canonical long-format tracking structure wherever possible.

Provider-specific raw files may differ significantly internally, but downstream engineering layers should operate on standardized processed outputs.

Recommended canonical tracking schema:

canonical_tracking_players

- provider
- match_id
- frame
- timestamp
- period
- team_id
- player_id
- player_name
- position_group
- role_name
- x_raw
- y_raw
- x_metric

- y_metric
- coordinate_system_raw
- is_detected
- is_extrapolated
- source_file

Recommended canonical ball schema:

canonical_tracking_ball

- provider
- match_id
- frame
- timestamp
- period
- ball_x_raw
- ball_y_raw
- ball_z_raw
- ball_x_metric
- ball_y_metric
- ball_z_metric
- coordinate_system_raw
- ball_is_detected
- source_file

Engineering notes:

- SkillCorner and Metrica use fundamentally different coordinate systems.
- SkillCorner uses metric coordinates centered at midfield.
- Metrica uses normalized coordinates approximately bounded around [0,1].
- Processed schemas must preserve both raw and normalized/metric coordinate representations where possible.
- Downstream tactical metrics should rely on standardized metric coordinates rather than provider-native raw coordinates.

2. Provider Normalization Rule

Initial ingestion must preserve provider-native raw values.

DO NOT overwrite or destroy provider-specific coordinate information during ingestion.

Required rule:

preserve raw coordinates

+

generate standardized metric coordinates separately where possible

Reasoning:

Future project stages may require:

- provider-specific debugging,
- coordinate validation,
- orientation correction,
- compactness calculations,
- centroid calculations,
- spatial-temporal modeling,
- and provider comparison workflows.

Destroying provider-native coordinates during ingestion would reduce future engineering flexibility and make debugging substantially harder.

3. Metrica Sample Game 3 Status

Metrica Sample_Game_3 has been discovered and structurally identified.

Validated files include:

- events.json
- metadata.xml
- tracking.txt

However:

Sample_Game_3 is NOT currently part of the first reusable ingestion implementation scope.

Current reusable ingestion development should focus on:

- Sample_Game_1
- Sample_Game_2
- SkillCorner Open Data

Reasoning:

Sample_Game_1 and Sample_Game_2 represent the standard Metrica tracking workflow most commonly used in football analytics environments.

Sample_Game_3 uses a separate internal structure and would require a separate provider-specific ingestion path.

Future integration of Sample_Game_3 may occur later if explicitly assigned.

DO NOT expand ingestion scope unnecessarily during the current Layer 1 engineering phase.

4. Processed Output Folder Convention

Reusable ingestion outputs should follow a standardized processed-data structure.

Recommended processed folder structure:

data/processed/skillcorner/

data/processed/metrica/

data/processed/validation_reports/

Possible future expansion folders:

data/processed/normalized/

data/processed/features/

Engineering notes:

- Raw provider data should remain untouched in external/ or raw-data folders.
 - Processed outputs should be reproducible from raw sources through ingestion pipelines.
 - Validation reports should be stored separately from processed tracking tables.
 - Provider separation should remain explicit during early architecture stages.
-

5. Shared Validation Report Schema

All provider ingestion pipelines should generate standardized validation summaries.

Recommended validation output structure:

```
{  
  "provider": "...",  
  "match_id": "...",  
  "raw_tracking_files_found": True,  
  "raw_event_files_found": True,  
  "tracking_total_frames": ...,  
  "usable_player_frames": ...,  
  "fps": ...,  
  "coordinate_system": "...",  
  "player_join_success": ...,  
  "event_alignment_success": ...,  
  "phase_alignment_success": ...,  
  "known_issues": [...]  
}
```

Engineering notes:

- Validation outputs should be machine-readable.
- Validation outputs should support automated ingestion diagnostics later.
- Provider-specific warnings should be preserved in known_issues.
- Validation reporting should remain lightweight and reproducible.